

a scoping issue, Link to the appropriate PS Operations handbook page to show the process in mavenlink.

Scheduling

The Professional Services (PS) team scheduling is processed through the Sr. PS Project Coordinator (PC). Our PSA system calendar is our single source of truth for scheduling our customer and internal projects.

Follow these steps to schedule a customer engagement.

Submit a Resource Request through Mavenlink with the following details:

- Role
- Dates to be worked
- Hours requested
- Soft or Hard allocation

The PC will review the master planning for availability and process the resource request. If there is a schedule conflict the PC will provide another set of project dates.

Scheduling updates and changes follow this same process with a resource request in Mavenlink.

If a customer project has not booked, but planning/scheduling discussions need to take place, reach out to the PC to review.

How to schedule internal time

There are 2 project that track internal time, Creditable and Non Creditable. If hours need to be scheduled for the projects, a comment in the project activity and mention the PC:

- Requested Dates
- Hours requested
- Task assignment
- Soft or Hard allocation

3. Engagement Execution

Kickoff

See the details in the Project workflow section of the PS Project Management page.

Plan

TODO: Add content about what happens in this step

Discovery

During discovery or fact finding sessions with the customer, PSEs will often have a predefined list of questions that need to be answered to ensure we're designing and building the

appropriate solution given customer constraints and requirements. It is good practice to send these questions to the customer prior to the discovery call so they can be prepared for the discussion.

During the call, take notes to ensure that things that have validated or invalidated your initial assumptions have been captured. At the end of the meeting, review the things you've learned to memorialize what will be designed and built. Reiterating back your understanding of the details of the agreement instills confidence in the customer representative that we understand their requirements and can deliver what was reviewed.

After the meeting, based on meeting notes, create issues in the gitlab.com customer collaboration project outlining the work. Include Consider using a simple template with Overview, Open Questions, Tasks, and Acceptance Criteria. These can be helpful in further memorializing the scope of work with the customer and getting asynchronous feedback to open questions. Make sure the overview is as detailed as possible, and the tasks section has build-to level tasks (e.g. update `congregate list()` function to include data from CI sources).

Build/Validate

The build validation step is a bit vague on purpose because it depends on what was included in the scope of the engagement. The PSE can use the delivery kits mapped to our service offerings which include templated discovery documents, automation software to facilitate service delivery, and templated deliverable documents. Most of the collateral will be modified and updated right from the delivery kits so this section is intentionally light on detail.

Train

PSEs or Technical Instructors who deliver GitLab Education Services instructor-led courses can use the following workflow to ensure smooth interactions with customers. In addition, PSEs and Technical Instructors should complete these GitLab Certified Trainer steps for each course they are scheduled to deliver.

Preparation steps

1. The Project Coordinator will contact the customer with a "Welcome to Education Services Email" to initiate the training scheduling. After the training dates and times are confirmed, the Project Coordinator will schedule a training session planning meeting. Trainer participation in this meeting is recommended -- please let the Project Coordinator know if you need the meeting to be rescheduled to ensure your attendance.

1. The Project Coordinator will use these email communication templates to ensure communication of the key details with the customer and training participants.

1. During the training planning meeting, be sure to discuss and document all of the event logistics listed in the Training Event Plan Template. The Project Coordinator will create a draft of the Training Event Plan prior to the meeting and update the document during the training planning meeting.

- During the training planning meeting, the course outline and system requirements pages below are useful to use to review training logistics, topics, teleconferencing, and system requirements.

- GitLab with Git Fundamentals course outline
- GitLab CI/CD course outline
- GitLab Agile Portfolio Management course outline
- GitLab Security Essentials course outline
- GitLab System Administration course outline
- GitLab Duo Principles course outline
- System Requirements

1. The Project Coordinator will set up a Zoom Meeting or Webinar session for each session using these set up instructions and add the registration link(s) to the issue. You will receive an email message with your unique link to join the Zoom Meeting or Webinar session. Make sure to locate the Zoom information within the email message and familiarize yourself with the Zoom functionality. Here is a useful Zoom article for Managing Attendees and Panelists in a Webinar. Depending on your Zoom set up, you may want to log into <https://zoom.us>, go to Join a Meeting, and enter the meeting ID/webinar ID to start the Zoom session.

1. At least 2 weeks prior to the training session, the Project Coordinator will email the session registration link(s) to the customer, asking them to send the link(s) to each of the employees whom they want to attend the session(s). When each person registers, they will receive an automated confirmation email with a Zoom Meeting or Webinar join link unique to each person, along with a link to add the session to their calendar.

1. The Project Coordinator will advise if a different teleconferencing system is being used for the training and provide additional details for accessing the teleconferencing meeting.

1. Contact the GitLab Education Services team to confirm you have the latest versions of course slides and other materials.

1. Review the train-the-trainer (T3) video for the course you are delivering.

1. Review and follow the Instructor Pre-Training Checklist to ensure that you are properly prepared for your delivery.

1. Review, practice, and use these PS Remote Training Tips and Tricks.

1. Complete the GitLab Training Lab set up steps below. Make sure that you review the lab exercises and ensure that the labs are working properly prior to the first day of class.

1. When it's time to join the teleconferencing as a presenter, use the provided information to join the session.

Training lab pre-course instructor workflow

PS uses the GitLab Lab Environment as the standard environment for hands-on course lab activities and hands-on certification assessments. Follow these steps to set up your course attendees for lab access.

1. GitLab Demo System invitation codes

1. The Project Coordinator will generate the invitation code for your class and provide the

invitation code information approximately a week prior to your class start date as part of the instructor friendly reminders post within your Kantata project.

1. Follow the instructions on the GitLab Lab Environment Invitation Code Redemption handbook page to redeem the invitation code and access the lab environment.

1. For any edits/extensions/etc or any custom redemption rules (different durations than our standards), contact the GitLab Professional Services Operations team for assistance.

2. Share the invitation code and access instructions with attendees:

1. On the first day of class, share the invitation code and review the login process with the attendees. Also let them know the expiration date (which is 30 days from the date they generate the login).

Training Closeout

1. Review the Instructor Post-Training Checklist to ensure that you have followed all of the steps to close out your training class.

1. The Project Coordinator will download the attendance report and send a close out email to the customer using an email template located in the email communication templates.

Complete

For blended engagements, see the Deploy & Close section of our [project management page]](/handbook/customer-success/professional-services-engineering/project-mgmt/project-workflow)

Financial Closeout

TODO: Add or link to content about what happens in this step

4. Customer Success

Retrospective

The project manager should schedule a retrospective at the end of the project. Invite the account team (SAE, AE, CSM, SA), the delivery team (PSEs, trainers, Project Coordinators), Engagement Manager who scoped the engagement, and PS Leadership team.

The Retrospective TEMPLATE can be used to capture notes during the meeting.

The Retrospective meeting should be run by someone that is not intimately familiar with the execution of the project (typically PS Practice Manager or Engagement Manager). During the meeting, encourage contributions into the notes doc in real time and offer for participants to verbalize their feedback.

Toward the end of the call, gather actions and assign owners to complete those actions.

Project Go-live/Recap Document

At the end of a project the Project leader should document the customer value drives/positive

business outcomes they were looking to accomplish, what PS delivered to help them accelerate their journey, and special thanks to the project team. A template of this write up be found in the PMO templates Google Drive folder. An example can be found here.

Success Plan (CSM)

TODO: Add or link to content about what happens in this step

Handoff (Support)

At the end of a small/medium sized projects or after the initial implementation phase of a large engagement, open a support ops issue and add a link to the collaboration project architecture diagram that was deployed. The Support team will load it into a field in Zendesk to help them have context for customer support requests. -->

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/agile-to-gitlab-terminology/_index.md

The following table and blog link show the agile to GitLab terminology mapping.

<table>

<tr>

<td>

Agile artifact

</td>

<td>

GitLab feature

</td>

</tr>

<tr>

<td>User story</td>

<td>

Issues

</td>

</tr>

<tr>

<td>Task</td>

<td>

Task lists

</td>

</tr>

<tr>

<td>Epic</td>

<td>

Epics
</td>
</tr>
<tr>
<td>Points and estimation</td>
<td>

Weights
</td>
</tr>
<tr>
<td>Product backlog</td>
<td>

Issue lists and prioritized labels
</td>
</tr>
<tr>
<td>Sprint/iteration</td>
<td>

Milestones
</td>
</tr>
<tr>
<td>Burndown chart</td>
<td>

Burndown charts
</td>
</tr>
<tr>
<td>Agile board</td>
<td>

Issue boards
</td>
</tr>
</table>

Blog: How to use GitLab for Agile Software Development

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/archetype-definition/_index.md

What Does a Large Customer Look Like? The Archetype

Most Fortune 500 companies are interested in Scaling. The business goals are usually focused on

gaining efficiencies by scaling from teams to programs and ultimately across a set of complex portfolios with a strong incentive to realize cost savings through standardization.

Some Fortune 500 companies are interested in digital transformations.

What are key characteristics?

Fortune 500
New logo, large Dedicated or Self Managed customer
Complex portfolio with many programs and distributed application teams
Geographically distributed office locations (for example: North East/NY, Houston/TX, Bangalore/India)
\$1M+ PS revenue
At least three (3) logical portfolio groupings
Looking to either commit to 500+ Ultimate licenses for new logo, or expand from current 1,000+ Ultimate licenses to x Ultimate licenses in the coming year
Source origination either from competitor (GitHub, BitBucket) or existing GitLab variety
Looking for total tool usage of GitLab across their Dev organization or for digital transformation and getting off a variety of legacy tooling
Customer has two (2) critical dependencies / seasonal events that prevents them from development work 3 weeks before and 3 weeks after each event (system freeze, minimizing system changes), such as
 Super Bowl
 Black Friday
 4th of July Sales Event

Scaling Considerations

- ☐ Company Size
- ☐ Locations / Time Zones
- ☐ Number of Portfolios / Programs / Teams
- ☐ Technical Environment
- ☐ DIY Tool Chain
- ☐ Legacy Systems
 - ☐ Jira
 - ☐ Jenkins
 - ☐ ADO
 - ☐ External Security Scanners (Fortify, Snyke, Black Duck, etc.)
 - ☐ Package Managers in Use
 - ☐ ... \<Action Item\>
- ☐ Number of Users
- ☐ Number of Transactions
- ☐ Data Volume
- ☐ Repository Size
- ☐ Repository Depth
- ☐ <Action Item>

[] <Action Item>

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/backlog-management/_index.md

One of the most frequent challenges in Agile implementation teams is with the proactive management of their Product and Sprint Backlogs.

Teams struggle with the process of creating good backlogs, maintaining relevant information in backlogs, and regularly refining them.

The problem arises from a combination of issues such as:

The Program Manager / Project Manager being overwhelmed or spread too thin

The lack of a clear vision that translates into a product roadmap

The lack of a product roadmap that translates into clearly expressed release goals

Insufficient tools expertise or lack of understanding Agile best practices

Problems estimating the backlogs in time

The following picture shows how these things are strongly related:

Prerequisites

To reiterate some of the basic preconditions that have to be met in order to effectively manage the Product and Sprint Backlogs, they are:

1. Good user stories
2. Good estimation techniques
3. 1 and 2 result in the ability to perform effective release planning.

Without good user stories and applying good estimation techniques, the product and sprint backlogs become less useful and will ultimately prevent the Agile / Scrum process from becoming predictable.

The Program Manager / Project Manager provides strategy and direction for the project, which means he/she is responsible for providing the vision, product roadmap, release goals, sprint goal. The Program Manager / Project Manager is expected to insert, re-prioritize, refine, or delete items from the product backlog; this can happen any time until the sprint scope is defined and committed to by the development team.

Sizing and estimation of the product backlog usually occurs in sprint planning meetings or at regularly intervals during ongoing sprints. Depending on how fast the Program Manager / Project

Manager adds user stories, more frequent, maybe even daily estimation sessions might be needed. The GitLab Implementation Team and the Customer Development Team and Program Manager / Project Manager must work together to estimate the backlog items, may that be in sprint planning meetings or during ongoing regular sessions.

The important point here is that product backlog items need to get estimated continuously in order to avoid the big bang estimation effort at one critical juncture of the effort. The frequency of the estimation effort is depending on your engagement development effort. This has the added benefit of exposing the entire team to the Program Manager / Project Managers' thinking, keeping everybody in sync and with an improved understanding of requested product features and functionality.

Sprint Planning meetings or more frequent estimation session need to follow established Agile Estimation best practices, meaning Product Backlog Items are estimated using either Estimation Poker or T-Shirt sizing.

The Program Manager / Project Manager is responsible for initiating requirements, creating user stories, and exerting scope control.

The GitLab Implementation Team and the Customer Development Team are responsible for estimating the backlog in close collaboration with the Program Manager / Project Manager. This is a team effort and it is critical that GitLab and Customer team members agree on scope.

The Program Manager / Project Manager communicates with stakeholders to determine the priorities and constantly refines the backlog.

In GitLab, there are dynamically generated issue lists which users can view to track their backlog. Labels can be created and assigned to individual issues, which then allows you to filter the issue lists by a single label or multiple labels. This allows for further flexibility. Priority labels can be used to also order the issues in those lists. See "How to Use CPR to Manage Engagements" for more information.

Product · Release · Sprint Backlogs

Many efforts require you to manage three backlogs:

1. The Product Backlog, which essentially is the requirements database for the engagement, expressed in user story format · exists for the lifetime of the engagement
2. The Release Backlog, which is the subset of functionality that will have to be delivered in a specific release (consisting of a set of sprints), according to the product roadmap and release plan · active for a specific release
3. The Sprint Backlog, which represents the work to be done for the upcoming sprint · active for the sprint duration

Labels can be used to tag user stories / issues.

Using such tagging lets you easily view relevant information about the specific user story and what state it is in.

Usually the next upcoming sprint is sized and estimated in detail using Estimation Poker, with the following two to three sprints and their contents being sized and estimated using either Estimation Poker or T-Shirt Sizing. Remember that once a sprint is committed to by the team, the scope should not change any longer as you are "in-flight" - changing stories "in-flight" causes unnecessary context switching, which is costly in terms of productivity. Future sprints can still be adjusted and items can be reshuffled as the Product Owner and Development Team agree on.

Product Backlog Items and Sprint Backlog Items may contain more than just user stories, but regardless what exactly is in the specific backlog, it is supposed to represent all work items that consume team capacity. This includes the actual user story describing the engagement feature to be implemented, defects, research, and other technical tasks.

Remember that the Sprint Backlog is supposed to be the most precise and fleshed out backlog due to its immediacy to actual delivery of a working product increment:

- Consists of committed Product Backlog Items negotiated between the team and the Product Owner during the Sprint Planning Meeting

- Scope commitment is fixed during Sprint Execution

- Initial tasks are identified by the team during Sprint Planning Meeting

- Team will discover additional tasks needed to meet the fixed scope commitment during Sprint execution

- Visible to the team

- Referenced during the Daily Scrum Meeting

- Used to measure Sprint Acceptance by the Program Manager / Project Manager based on the Definition of Done

Backlogs are the Fulcrum of Agile / Scrum

"Garbage in, garbage out", in the case of Agile backlogs, it is true.

The single most important artifact impacting the delivery success of the current sprint is the Sprint Backlog.

The single most important artifact impacting the success or failure of your engagement / project is your Product Backlog.

Agile is a simple process, with 6 roles, 5 events, and 9 artifacts.

Only 8 artifacts are shown above - the working system left behind is implicitly considered the 9th.

Backlogs hold the essence of what needs to get implemented, and as such need to be created, maintained and groomed with great care.

Unless the Program Manager / Project Manager, the GitLab Implementation Team and the Customer Development Team focus on keeping backlogs up to date and well refined, the Agile process will not be able to deliver what is most important to organizations: Predictability.

Common Issues and Challenges

The section addresses some common issues and challenges that make managing product and sprint backlogs more difficult than it has to be.

Poorly Written User Stories

User stories should follow the INVEST principle. They should be:

- Independent (to the degree possible, user stories should not rely on other user stories to be implemented)

- Negotiable

- Valuable (meaning they are feature focused, not task oriented, and are written in the user's language to the extent possible; where domain specific language is required, such as in genomics or medicine, a subject matter expert will have to be available)

- Estimable

- Small (half a Sprint or less for one Dev Team member)

- Testable (needs to have acceptance criteria and not be subjective)

The testability of a user story is usually documented in the form of Acceptance Criteria and lists pass/fail testable conditions that help us verify if the user story is implemented as intended.

All sounds simple enough, but frequently teams and Program Manager / Project Manager really struggle with writing good user stories, and 90% of all problems encountered are based on user stories being too big.

When a user story is too big, it is hard understand, estimate, implement, and prone to wildly different opinions. So what is a good size for a user story? Basic guidance is that the user stories at the top of the product backlog (the ones that should have sufficient specificity in order to be worked on by a developer) should be sized such that a team member could easily complete two user stories during a sprint.

Good user stories are the basis for good backlogs.

Rapid Requirements Inflow

Sometimes the Program Manager / Project Manager might start adding user stories to the backlog far faster than the Development Team can estimate the effort.

Or, the Program Manager / Project Manager, out of desperation because the Implementation and Development Teams are busy with implementation tasks, starts estimating the effort himself/herself, without the technical teams being involved. This is a bad idea!

One of the core Agile principles is that "business people and developers must work together daily", as such the Program Manager / Project Manager cannot simply hand over user stories to the technical teams as it defeats the purpose of jointly understanding and estimating user stories.

If the inflow of requirements outpaces the teams' ability to understand the requirements in detail and estimate them accordingly, the best course of action is to set up additional backlog refinement sessions to get things back on course.

Fantasy Wish List

The backlogs need to contain user stories that are relevant to GitLab, the engagement, the release, and the sprints. Cluttering the backlogs with fantasy requirements that do not directly contribute to the engagement, release, or sprint goals is counterproductive.

Such wish list items need to be kept separate and managed outside of the backlogs, until such requirements become tangible goals at the engagement, release or sprint level. Providing a label such as "Future" allows project participants to log desired issues while keeping them out of the actionable sprint scopes.

Backlogs = Requirements Specifications

Especially during Agile Transformation / Adoption efforts, teams often times deal with a long tail of old artifacts available from previous software development methodologies.

For example, customers might have many UML diagrams, functional or technical requirements specifications, etc. Why not put them to good use and just make them part of the user stories?

The reason not to do this is because specification have the side effect of shutting down discussions, as in "this is the specification we need to implement, nothing more, nothing less" · essentially specification hinder the discover of newly emerging requirements.

However, this is not to say that once a user story is well understood by the team, you will never, ever, write a technical specification document in order to document the implementation.

Unavailable Program Manager / Project Manager

One of the most common challenges with Program Manager / Project Manager is their lack of availability to work with the GitLab Implementation and Customer Development Team. Program Manager / Project Manager are enablers, making the technical teams more productive · or less!

Without an available Program Manager / Project Manager, the technical teams will get stuck · this might manifest itself either in sprint goals not being met, in poor quality, low velocity, lack of stakeholder management, delayed status reports, etc.

Lack of Backlog Refinement

There really is no excuse not to regularly refine and estimate your backlog. As mentioned before, if for whatever reason your backlog is not being refined, the best course of action is to set up additional backlog grooming sessions to get things back on course.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/cp/_index.md

What is CP?

the CP or "Collaboration Project" is the framework for how we would like to manage delivering engagements in a standardized, repeatable manner all within GitLab. Each customer will have their own group in GitLab and that group will contain a project for each SOW we deliver for that customer. These groups and projects will house all the information around the project delivery to easily track the progress and health of a project as well as provide a paper trail for all the work we have done during the project.

A list of our Projects can be found here: GitLab Professional Services Group if staffed internally or GitLab Partner Collaboration Group if staffed fully or partially with partners.

How does an SOW flow into CP?

A PSQuote quote contains one to many quote lines. Those quote lines map 1:1 to epics within a customer group on gitlab.com and each epic is then treated as a workstream. That customer group will contain one to many projects depending on how many SOWs we deliver for that customer. For each epic, the PM will create various issues tracking specific work towards the completion of each workstream.

What are the common components?

- A customer group: This group will house all the work we do for a specific customer.
- An SOW/PO project: These projects will reside within the customer group and house all the work we do within that specific SOW or PO.
- Workstream epics at the customer group label: These are what tie directly to the activities in the SOW.
- A standard library of issue templates.
- A standard library of comment templates.
- A standard series of boards covering the status of each task/issue per workstream.
- Any specific documentation from our delivery kits.

How will all this get created?

We have a project called CPRGitOps that handles the process of automatically creating the foundation of groups, projects, epics, etc per Customer Project. This automation is triggered by our Quilt API through PSQuote. We are currently using a merge request as the staging area to provide all the necessary details CPRGitOps needs to properly scaffold out a project to deliver.

Please follow this README to get your project setup within GitLab while we continue to make this step more automated or send a message to the ps-practice slack channel, requesting a CP project to be created.

Example CP Project-Delta (Migration)

Example CP Project-Ripple (Migration)

What does the PM need to configure once the Project is ready?

- issue board
- readme

How far we've come

Customer Project (CP) automation was an effort initially designed back in 2022 by Adriano Fonseca as an OKR to improve our delivery processes. The original design can be found [here](#). This design has changed a bit as we have worked to revive the effort.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/definition-of-done/_index.md

The Definition of Done and the Definition of Ready are closely related. Or the other way around; it is a chicken and egg thing.

The Big Picture

One of the key artifacts of Agile is the idea of backlogs · both at the Product level as well as at the Sprint level.

The Product Backlog is essentially the project's To-Do list, or requirements repository.

All items that are deemed in scope for the project, regardless of level of detail, are in the Product Backlog, and they are ordered, not just prioritized · meaning the one on top is more important than the one in 5th position, which is more important than the one in 23rd position. Order is decided by the Program Manager / Product Manager and usually driven by business value · in consultation with the Customer Development Team.

What is known of the projects scope, is written down and documented in the form of user stories, with the expectation that discovery will lead to change. The Product Backlog is a living repository and is owned by the Program Manager / Project Manager.

The Product Backlog is the source for the Sprint Backlog. Whereas the Product Backlog represents the requirements repository for the project / engagement, the Sprint Backlog is the agreed upon scope for the next upcoming sprint and as such represents the GitLab Implementation and Customer Development Team's deliverable commitment. The GitLab Implementation Team does not operate in a vacuum, it close coordinates priorities with the Customer Development Team.

Once agreed upon and committed to, the Sprint Backlog usually does not change in order to ensure the GitLab Implementation Team and the Customer Development Team can deliver against

their commitment.

The Implementation Team and Development Team works through the Sprint Backlog top to bottom (most important to least important) and if it runs out of work would usually look at the Product Backlogs' top items in order to pull additional work into the current sprint.

If items are not finished during the current sprint, then they are a) considered for implementation in the next sprint, or b) reverted back to the Product Backlog to be reconsidered for the next sprint in the upcoming Sprint Planning Meeting.

Newly discovered requirements are added to the Product Backlog in the form of user stories, to be considered for future sprints.

Definition of Ready (DoR) vs. Definition of Done (DoD)

A sprint is a time-boxed development cycle that takes high-priority items off the Sprint Backlog and turns them into a product increment. However, in order to successfully pull items into the current sprint, it is important that the defined user stories are "ready" · pulling unfinished or unrefined user stories into a sprint causes problems during the implementation cycle, as it follows the old principle of "garbage in, garbage out". If developers work off of insufficiently detailed or defined user stories, they are unlikely to produce high quality code.

A "ready" backlog item needs to be clear, feasible and testable:

A user story is clear if all team members (GitLab Implementation Team and Customer Development Team) have a shared understanding of what it means. Collaboratively writing user stories, and adding acceptance criteria to the high-priority ones facilitates clarity

An item is testable if there is an effective way to determine if the functionality works as expected. Acceptance criteria ensure that each story can be tested

A user story is feasible if it can be completed in one sprint, according to the Definition of Done. If this is not achievable, it needs to be broken down further

Simply stated, the Definition of Ready defines the criteria that a specific user story has to meet before being considered for estimation or inclusion into a sprint. Stories that are not market READY should not be targeted in a sprint as clarification is needed before starting work.

Whereas a Definition of Ready is focused on user story level characteristics, the Definition of Done is focused on the sprint or release level.

Essentially, a DoD represents the acceptance criteria for a sprint or release. It spells out what the Implementation and Development Team has to cover in order for the product increment to be considered "done".

The Definition of Done is an agreement between GitLab Implementation Team, the Customer Development Team and the Program Manager / Project Manager on what needs to be completed for each user story · and it is often standardized across the customer engagement in order to guarantee consistent delivery of quality.

Things that commonly addressed in the Definition of Done are:

Operating environments and at what level of integration are user stories expected to work?
What level of documentation is required?
What are the quality expectations?
What are the security expectations?
Scalability expectations?

Essentially the Definition of Done are the agreed upon acceptance criteria that the Program Manager / Project Manager will use to accept the product increment at the end of the sprint. It is critical for consistent status reporting as well.

Please note that the DoD may be different for sprints vs. releases, meaning intermediate sprints might have a less stringent DoD than the final couple of sprints before you are planning to release to market. This is critical during engagements where initial sprints might produce early versions of the deliverables and later sprints will harden those deliverables.

Sample Definition of Ready (DoR)

- User Story is clear
- User Story is testable
- User Story is feasible
- User Story defined
- User Story Acceptance Criteria defined
- User Story dependencies identified
- User Story sized by Development Team
- Scrum Team accepts User Experience artefacts
- Performance criteria identified, where appropriate
- Scalability criteria identified, where appropriate
- Security criteria identified, where appropriate
- Person who will accept the User Story is identified
- Team has a good idea what it will mean to Demo the User Story

Sample Definition of Done (DoD)

- Code produced (all 'to do' items in code completed)
- Code commented, checked in and run successfully
- Peer reviewed (or produced with pair programming) and meeting development standards
- Builds without errors
- Unit tests written and passing - as appropriate
- Passed Acceptance Testing by the Customer Development Team and signed off as meeting requirements
- Any build / deployment / configuration changes are implemented / documented / communicated from sprint to sprint
- Relevant documentation / diagrams produced and / or updated
- Remaining hours for task set to zero and task closed

Other uses of the Definition of Ready and Definition of Done

The original intent of DoR and DoD was to create brief documents representing internal agreements between the projects' stakeholders, the Program Manager / Project Manager, the GitLab Implementation Team, and the Customer Development Team.

However, with more development work being outsourced or subcontracted, the DoR and DoD are now used more and more in contract agreements and statements of work, spelling out exact expectations as to what needs to be done - this is especially useful when setting expectations with partner consultants.

These are useful tools for negotiating project scope as they define expectations and hold all parties accountable; the DoR helps the customer producing well written user stories that are ready to be consumed by the Implementation and Development Teams, and the DoD helps the teams to produce working product increments according to all project requirements, not just the specific user story functionality.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/definition-of-ready/_index.md

The Definition of Ready and the Definition of Done are closely related. Or the other way around; it is a chicken and egg thing.

The Big Picture

One of the key artifacts of Agile is the idea of backlogs · both at the Product level as well as at the Sprint level.

The Product Backlog is essentially the project's To-Do list, or requirements repository.

All items that are deemed in scope for the project, regardless of level of detail, are in the Product Backlog, and they are ordered, not just prioritized · meaning the one on top is more important than the one in 5th position, which is more important than the one in 23rd position. Order is decided by the Program Manager / Product Manager and usually driven by business value · in consultation with the Customer Development Team.

What is known of the projects scope, is written down and documented in the form of user stories, with the expectation that discovery will lead to change. The Product Backlog is a living repository and is owned by the Program Manager / Project Manager.

The Product Backlog is the source for the Sprint Backlog. Whereas the Product Backlog represents the requirements repository for the project / engagement, the Sprint Backlog is the agreed upon scope for the next upcoming sprint and as such represents the GitLab Implementation and Customer Development Team's deliverable commitment. The GitLab Implementation Team does not operate in a vacuum, it close coordinates priorities with the Customer Development Team.

Once agreed upon and committed to, the Sprint Backlog usually does not change in order to ensure the GitLab Implementation Team and the Customer Development Team can deliver against their commitment.

The Implementation Team and Development Team works through the Sprint Backlog top to bottom (most important to least important) and if it runs out of work would usually look at the Product Backlogs' top items in order to pull additional work into the current sprint.

If items are not finished during the current sprint, then they are a) considered for implementation in the next sprint, or b) revert back to the Product Backlog to be reconsidered for the next sprint in the upcoming Sprint Planning Meeting.

Newly discovered requirements are added to the Product Backlog in the form of user stories, to be considered for future sprints.

Definition of Ready (DoR) vs. Definition of Done (DoD)

A sprint is a time-boxed development cycle that takes high-priority items off the Sprint Backlog and turns them into a product increment. However, in order to successfully pull items into the current sprint, it is important that the defined user stories are "ready" · pulling unfinished or unrefined user stories into a sprint causes problems during the implementation cycle, as it follows the old principle of "garbage in, garbage out". If developers work off of insufficiently detailed or defined user stories, they are unlikely to produce high quality code.

A "ready" backlog item needs to be clear, feasible and testable:

A user story is clear if all team members (GitLab Implementation Team and Customer Development Team) have a shared understanding of what it means. Collaboratively writing user stories, and adding acceptance criteria to the high-priority ones facilitates clarity

An item is testable if there is an effective way to determine if the functionality works as expected. Acceptance criteria ensure that each story can be tested

A user story is feasible if it can be completed in one sprint, according to the Definition of Done. If this is not achievable, it needs be broken down further

Simply stated, the Definition of Ready defines the criteria that a specific user story has to meet before being considered for estimation or inclusion into a sprint. Stories that are not market READY should not be targeted in a sprint as clarification is needed before starting work.

Whereas a Definition of Ready is focused on user story level characteristics, the Definition of Done is focused on the sprint or release level.

Essentially, a DoD represents the acceptance criteria for a sprint or release. It spells out what the Implementation and Development Team has to cover in order for the product increment to be considered "done".

The Definition of Done is an agreement between GitLab Implementation Team, the Customer Development Team and the Program Manager / Project Manager on what needs to be completed for each user story · and it is often standardized across the customer engagement in order to guarantee consistent delivery of quality.

Things that commonly addressed in the Definition of Done are:

Operating environments and at what level of integration are user stories expected to work?

What level of documentation is required?
What are the quality expectations?
What are the security expectations?
Scalability expectations?

Essentially the Definition of Done are the agreed upon acceptance criteria that the Program Manager / Project Manager will use to accept the product increment at the end of the sprint. It is critical for consistent status reporting as well.

Please note that the DoD may be different for sprints vs. releases, meaning intermediate sprints might have a less stringent DoD than the final couple of sprints before you are planning to release to market. This is critical during engagements where initial sprints might produce early versions of the deliverables and later sprints will harden those deliverables.

Sample Definition of Ready (DoR)

- User Story is clear
- User Story is testable
- User Story is feasible
- User Story defined
- User Story Acceptance Criteria defined
- User Story dependencies identified
- User Story sized by Development Team
- Scrum Team accepts User Experience artefacts
- Performance criteria identified, where appropriate
- Scalability criteria identified, where appropriate
- Security criteria identified, where appropriate
- Person who will accept the User Story is identified
- Team has a good idea what it will mean to Demo the User Story

Sample Definition of Done (DoD)

- Code produced (all 'to do' items in code completed)
- Code commented, checked in and run successfully
- Peer reviewed (or produced with pair programming) and meeting development standards
- Builds without errors
- Unit tests written and passing - as appropriate
- Passed Acceptance Testing by the Customer Development Team and signed off as meeting requirements
- Any build / deployment / configuration changes are implemented / documented / communicated from sprint to sprint
- Relevant documentation / diagrams produced and / or updated
- Remaining hours for task set to zero and task closed

Other uses of the Definition of Ready and Definition of Done

The original intent of DoR and DoD was to create brief documents representing internal agreements between the projects' stakeholders, the Program Manager / Project Manager, the GitLab Implementation Team, and the Customer Development Team.

However, with more development work being outsourced or subcontracted, the DoR and DoD are now used more and more in contract agreements and statements of work, spelling out exact expectations as to what needs to be done - this is especially useful when setting expectations with partner consultants.

These are useful tools for negotiating project scope as they define expectations and hold all parties accountable; the DoR helps the customer producing well written user stories that are ready to be consumed by the Implementation and Development Teams, and the DoD helps the teams to produce working product increments according to all project requirements, not just the specific user story functionality.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/discovery/_index.md

How Will Discovery be Conducted?

- ☐ Discovery Interviews
- ☐ Open / Anonymous Surveys
- ☐ Technical Architecture Review
- ☐ Value Stream Assessment
- ☐ ...
- ☐ ...
- ☐ ...

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/gitlab-best-practices/_index.md

This section provides a comprehensive overview of best practices in GitLab, highlighting key guidelines for efficient and effective usage.

The documentation provides recommendations based on standards defined by the National security agency \[NSA\] along with Cybersecurity and Infrastructure Security Agency \[CISA\] for organizations to standardize and strengthen the security of their CI/CD pipelines.

This article can be referred to for relevant information on the defined standards.

1. Use feature branches rather than direct commits on the main branch

Using feature branches is a simple way to develop and keep the source code clean. If a team has recently transitioned to Git from SVN, for example, they'll be used to a trunk-based workflow. When using Git, developers should create a branch for anything they're working on so that contributors can easily start the code review process before merging.

2. Test all commits, not only ones on the main branch

Some developers set up their CI to only test what has been merged into the main branch, but

this is too late in the software development life cycle, and everyone - from developers to product managers - should feel confident that the main branch always has green tests. It's inefficient for developers to have to test main before they start developing new features.

Include Application security scans like SAST, Secret Detection, Dependency Scanning, Container Scanning etc in the feature branches for each commit

3. Run every test on all commits. (If tests run longer than 5 minutes, they can run in parallel)

When working on a feature branch and adding new commits, run tests right away. If the tests are taking a long time, try running them in parallel. Do this server-side in merge requests, running the complete test suite. If there is a test suite for development and another only for new versions, it's worthwhile to set up \[parallel\] tests and run them all.

You can also have these displayed in each Merge Request.

{width="624" height="220"}

4. Perform code reviews before merging into the main branch

Don't test everything at the end of a week or project. Code reviews should take place as soon as possible, because developers are more likely to identify issues that could cause problems later in the life cycle. Since they'll find problems earlier, they'll have an easier time creating solutions.

GitLab code review workflow

The GitLab code review workflow is simple.

1. Push your changes to GitLab using a terminal or the GitLab interface,
2. Open a merge request from your terminal or the GitLab interface,
3. Request reviewers to approve your merge request,
4. (optional) If you have set up CI/CD automation, wait for the green light before merging,
5. Reviewers will approve your merge request or request changes, change your code accordingly,
6. If all required reviewers have approved the merge request, you can now safely merge if there is no conflict with the base branch.

Code Review best practices:

1. A merge request should be first reviewed by a reviewer in each category (for example: backend, database) the MR touches, as maintainers may not have the relevant domain knowledge. This also helps to spread the workload. Add approval rules for each Code review. For example:
 1. Backend approvers
 2. Frontend approvers
 3. Database approvers
 4. Documentation
 5. Etc
2. For assistance with security scans or comments, include the Application Security Team
3. The reviewers use the reviewer functionality in the sidebar. Reviewers can add their

approval by approving additionally.

4. Depending on the areas your merge request touches, it must be approved by one or more maintainers. The Approved button is in the merge request widget.

5. Getting your merge request merged also requires a maintainer. If it requires more than one approval, the last maintainer to review and approve merges it.

6. Using labels in MR helps to identify the present status of the MR whether it is in code-review/work in-progress or anything else.

7. Below are some tips and guidelines to follow

Having your merge request reviewed

Please keep in mind that code review is a process that can take multiple iterations, and reviewers may spot things later that they may not have seen the first time.

The first reviewer of your code is you. Before you perform that first push of your shiny new branch, read through the entire diff. Does it make sense? Did you include something unrelated to the overall purpose of the changes? Did you forget to remove any debugging code?

Write a detailed description as outlined in the merge request guidelines. Some reviewers may not be familiar with the product feature or area of the codebase. Thorough descriptions help all reviewers understand your request and test effectively.

If you know your change depends on another being merged first, note it in the description and set a merge request dependency.

Be grateful for the reviewer's suggestions. ("Good call. I'll make that change.")

Don't take it personally. The review is of the code, not of you.

Explain why the code exists. ("It's like that because of these reasons. Would it be more clear if I rename this class/file/method/variable?")

Extract unrelated changes and refactorings into future merge requests/issues.

Seek to understand the reviewer's perspective.

Try to respond to every comment.

The merge request author resolves only the threads they have fully addressed. If there's an open reply, an open thread, a suggestion, a question, or anything else, the thread should be left to be resolved by the reviewer.

It should not be assumed that all feedback requires their recommended changes to be incorporated into the MR before it is merged. It is a judgment call by the MR author and the reviewer as to if this is required, or if a follow-up issue should be created to address the feedback in the future after the MR in question is merged.

Push commits based on earlier rounds of feedback as isolated commits to the branch. Do not squash until the branch is ready to merge. Reviewers should be able to read individual updates based on their earlier feedback.

Request a new review from the reviewer once you are ready for another round of review. If you do not have the ability to request a review, @ mention the reviewer instead.

Requesting a review

When you are ready to have your merge request reviewed, you should request an initial review by selecting a reviewer based on the approval guidelines.

When a merge request has multiple areas for review, it is recommended you specify which area a reviewer should be reviewing, and at which stage (first or second). This will help team members who qualify as a reviewer for multiple areas to know which area they're being requested to review. For example, when a merge request has both backend and frontend concerns, you can

mention the reviewer in this manner: @Johndoe can you please review \~backend? or @JaneDoe - could you please give this MR a \~frontend maintainer review?

You can also use workflow::ready for review label. That means that your merge request is ready to be reviewed and any reviewer can pick it. It is recommended to use that label only if there isn't time pressure and make sure the merge request is assigned to a reviewer.

When your merge request receives an approval from the first reviewer it can be passed to a maintainer. You should default to choosing a maintainer with domain expertise, and otherwise follow the Reviewer Roulette recommendation or use the label ready for merge.

Sometimes, a maintainer may not be available for review. They could be out of the office or at capacity. You can and should check the maintainer's availability in their profile. If the maintainer recommended by the roulette is not available, choose someone else from that list.

It is the responsibility of the author for the merge request to be reviewed. If it stays in the ready for review state too long it is recommended to request a review from a specific reviewer.

Reviewing a merge request

Understand why the change is necessary (fixes a bug, improves the user experience, refactors the existing code). Then:

- Try to be thorough in your reviews to reduce the number of iterations.

- Communicate which ideas you feel strongly about and those you don't.

- Identify ways to simplify the code while still solving the problem.

- Offer alternative implementations, but assume the author already considered them. ("What do you think about using a custom validator here?")

- Seek to understand the author's perspective.

- Check out the branch, and test the changes locally. You can decide how much manual testing you want to perform. Your testing might result in opportunities to add automated tests.

- If you don't understand a piece of code, say so. There's a good chance someone else would be confused by it as well.

- Ensure the author is clear on what is required from them to address/resolve the suggestion.

- Consider using the Conventional Comment format to convey your intent.

- For non-mandatory suggestions, decorate with (non-blocking) so the author knows they can optionally resolve within the merge request or follow-up at a later stage.

- There's a Chrome/Firefox add-on which you can use to apply Conventional Comment prefixes.

- Ensure there are no open dependencies. Check linked issues for blockers. Clarify with the authors if necessary. If blocked by one or more open MRs, set an MR dependency.

- After a round of line notes, it can be helpful to post a summary note such as "Looks good to me", or "Just a couple things to address."

- Let the author know if changes are required following your review.

If the merge request is from a fork, also check the additional guidelines for community contributions.

Merging a merge request

Before taking the decision to merge:

- Set the milestone.

- Confirm that the correct MR type label is applied.

Consider warnings and errors from danger bot, code quality, and other reports. Unless a strong case can be made for the violation, these should be resolved before merging. A comment must be posted if the MR is merged with any failed job.

If the MR contains both Quality and non-Quality-related changes, the MR should be merged by the relevant maintainer for user-facing changes (backend, frontend, or database) after the Quality related changes are approved by a Software Engineer in Test.

At least one maintainer must approve an MR before it can be merged. MR authors and people who add commits to an MR are not authorized to approve the MR and must seek a maintainer who has not contributed to the MR to approve it. In general, the final required approver should merge the MR.

Scenarios in which the final approver might not merge an MR:

- Approver forgets to set auto-merge after approving.
- Approver doesn't realize that they are the final approver.
- Approver sets auto-merge but it is un-set by GitLab.

If any of these scenarios occurs, an MR author may merge their own MR if it has all required approvals and they have merge rights to the repository. This is also in line with the GitLab bias for action value.

This policy is in place to satisfy the CHG-04 control of the GitLab Change Management Controls.

To implement this policy in gitlab-org/gitlab, we have enabled the following settings to ensure MRs get an approval from a top-level CODEOWNERS maintainer:

- Prevent approval by author.
- Prevent approvals by users who add commits.
- Prevent editing approval rules in merge requests.
- Remove all approvals when commits are added to the source branch.

To update the code owners in the CODEOWNERS file for gitlab-org/gitlab, follow the process explained in the code owners approvals handbook section.

Some actions, such as rebasing locally or applying suggestions, are considered the same as adding a commit and could reset existing approvals. Approvals are not removed when rebasing from the UI or with the /rebase quick action.

When ready to merge:

Consider using the Squash and merge feature when the merge request has a lot of commits. When merging code, a maintainer should only use the squash feature if the author has already set this option, or if the merge request clearly contains a messy commit history, it will be more efficient to squash commits instead of circling back with the author about that. Otherwise, if the MR only has a few commits, we'll be respecting the author's setting by not squashing them. Go to the merge request's Pipelines tab, and select Run pipeline. Then, on the Overview tab, enable Auto-merge. Note that:

If the default branch is broken, do not merge the merge request except for very specific cases. For other cases, follow these handbook instructions.

If the latest pipeline was created before the merge request was approved, start a new pipeline to ensure that the full RSpec suite has been run. You may skip this step only if the merge request does not contain any backend change.

If the latest merged results pipeline was created less than 6 hours ago, and finished less than 2 hours ago, you may merge without starting a new pipeline as the merge request is close enough to main.

When you set the MR to auto-merge, you should take over subsequent revisions for anything that would be spotted after that.

For merge requests that have had Squash and merge set, the squashed commit's default commit message is taken from the merge request title. You're encouraged to select a commit with a more informative commit message before merging.

Thanks to merged results pipelines, authors no longer have to rebase their branch as frequently anymore (only when there are conflicts) because the Merge Results Pipeline already incorporates the latest changes from main. This results in faster review/merge cycles because maintainers don't have to ask for a final rebase: instead, they only have to start a MR pipeline and set auto-merge. This step brings us very close to the actual Merge Trains feature by testing the Merge Results against the latest main at the time of the pipeline creation.

5. Deployments are automatic based on branches or tags

If developers don't want to deploy main every time, they can create a production branch. Rather than using a script or doing it manually, teams can use automation or have a specific branch that triggers a production deploy.

6. Tags are set by the user, not by CI

Developers should use tags so that the CI will perform an action rather than having the CI change the repository.

7. Releases are based on tags

Each tag should create a new release. This practice ensures a clean, efficient development environment.

8. Pushed commits are never rebased

When pushing to a public branch, developers shouldn't rebase it, because that makes it difficult to identify the improvement and test results, while cherry picking. Sometimes this tip can be ignored when asking someone to squash and rebase at the end of a code review process to make something easier to revert. However, in general, the guideline is: Code should be clean, and history should be realistic.

9. Everyone starts from main and targets main

This tip prevents long branches. Developers check out main, build a feature, create a merge request, and target main again. They should do a complete review before merging and eliminating any intermediate stages.

10. Fix bugs in main first and release branches second

After identifying a bug, a problematic action someone could take is fix it in the just-released version and not fix it in main. To avoid it, developers should always fix forward by pushing the change in main, then cherry-pick it into another patch-release branch.

11. Commit messages reflect intent

Developers should not only say what they did, but also why they did it. An even more useful tactic is to explain why this option was selected over others to help future contributors understand the development process. Writing descriptive commit messages is useful for code reviews and future development.

12. CI best practice: Optimize pipeline stages

CI pipelines contain jobs and stages: Jobs are the activities that happen within a particular stage, and once all jobs pass, code moves to the next stage. To get the most out of your CI pipelines, optimize stages so that failures are easy to identify and fix.

Stages are an easy way to organize similar jobs, but there may be a few jobs in your pipeline that could safely run in an earlier stage without negatively impacting your project if they fail. Consider running these jobs in an earlier stage to speed up CI pipelines.

13. CI best practice: Test environment should mirror production

In continuous integration, every commit triggers a build. These builds then run tests to identify if something will be broken by the code changes you introduce. The test pyramid is a way for developers to think of how to balance testing. End-to end testing is mostly used as a safeguard, with unit testing being used most often to identify errors. One important thing to keep in mind with testing is the environment. When the testing and production environments match, it means that developers can rely on the results and deploy with confidence.

In GitLab, Review Apps put the new code into a production-like live environment to visualize code changes. This feature helps developers assess the impact of changes.

Continuous integration helps developers deploy faster and get feedback sooner. Ultimately, the best continuous integration system is the one you actually use. Find the right CI for your needs and then incorporate these best practices to make the most of your new CI workflow.

14. CI best practice: Keep it safe

CI/CD is a shift left, so it offers a good opportunity to integrate security earlier in the process

15. Integrations with third-party

If JIRA is used, integrate Jira with GitLab to get timely updates on builds/branches/MRs

- Add a rule to add JIRA ticket in every MR commit

Teams-GitLab integration to send critical notifications like

- Prod/Pre-prod deployment failure

16. Protected Branches

A protected branch controls:

Which users can merge into the branch.

Which users can push to the branch.

If users can force push to the branch.

If changes to files listed in the CODEOWNERS file can be pushed directly to the branch.

Which users can unprotect the branch.

The default branch for your repository is protected by default.

17. Environment Branching strategies

It might be a good idea to have an environment that is automatically updated to the staging branch. Only, in this case, the name of this environment might differ from the branch name. Suppose you have a staging environment, a pre-production environment, and a production environment:

{width="356" height="340"}

In this case, deploy the staging branch to your staging environment. To deploy to pre-production, create a merge request from the staging branch to the pre-prod branch. Go live by merging the pre-prod branch into the production branch. This workflow, where commits only flow downstream, ensures that everything is tested in all environments.

18. Compliance frameworks

It is important for compliance teams to be confident that their controls and requirements are set up correctly, but also that they stay set up correctly. To obtain this confidence, compliance pipelines can be configured.

A compliance officer will be responsible for creating and enforcing the usage of a pipeline. We can ensure that a developer cannot change a running pipeline. This is a task that can only be performed by a compliance officer, ensuring only compliant code can be pushed without approval.

19. Security policies

GitLab provides Security Policies, which enable security teams to require security scans to run according to a configuration. This provides the security teams with confidence that the configured scans have not been changed or disabled.

There are two types of policies, Scan Execution Policies and Merge Request Approval Policies.

20. Audit Management and Compliance Dashboard

Another important part of compliance is knowing it is actually happening in your groups/projects. GitLab has Audit Events and Compliance Reports to assist with audits.

Audit Events allows GitLab owners and administrators to track important events such as who performed certain actions and the time they occurred.

{width="496" height="322"}

Audit Events records different events per group and per project, which can be seen in the audit events documentation. Audit Events can be accessed by going to Security & Compliance \> Audit Events Some examples include:

- user was added to project and their permissions
- permission changes of a user assigned to a project
- project CI/CD variable added, removed, or protected status changed
- user was added to group and their permissions
- group name or path changed

Audit Events can also be sent to an HTTP endpoint using Audit Event Streaming. I show how to implement Audit Event Streaming in this video.

Compliance Report gives you the ability to see a group's merge request activity. It provides a high-level view for all projects in the group.

{width="524" height="339"}

You can use the report to:

- get an overview of the latest merge request for each project
- see if merg